CWC 2.0 ESM — Data Model

Source: `backend/prisma/schema.prisma` (210 models total). This doc covers the **ESM service-management domain** — the request platform, service desks, workflow engine, approvals, SLA, notifications, assets, and knowledge base. Credit and CRM models are excluded (see `docs/handover/` and CRM docs).

Conventions:

- `RequestStatus` is the master status enum (~110 values, see §3); `RequestStatusLifecycleType` (OPEN/RESOLVED/CLOSED/CANCELLED) classifies lifecycle stage.
- Many entities use soft delete (`deletedAt`); query with `deletedAt: null`.
- Table names snake_case (`@@map`), UUID PKs with `@db.Uuid`.
- God-models: `Request` (~40 fields / 40+ relations), `User` (~163 fields / ~110 relations).

1. Domain map of ESM models

(a) Service desks & catalog

Model	Purpose / key FKs
`ServiceDesk`	A desk (IT / HR / Finance). Has `autoAssignTeam`, `assignmentStrategy` (ROUND_ROBIN), `lastAssignedIndex`, `autoAssignUser` (fixed assignee). Children: `ServiceCategory[]`, `Request[]`, `KbArticle[]`
`ServiceCategory`	Category within a desk (IT has ~5, HR ~4, Finance ~3). Relation: `serviceDeskId`
`RequestType`	A requestable service within a category. Has `requiresApproval`, `slaHours`, `formConfig` (JSON form schema), `formConfigVersion`, `workflowTypeId` (links to WorkflowType), `requiredRole`, `lifecycleStatus` (DRAFT/ACTIVE...), `classification` (INTERNAL/...). FKs: `serviceCategoryId`, `workflowTypeId`, `ownerId`

(b) Request lifecycle

Model	Purpose / key FKs
`Request`	**Central entity.** Fields: tenantId, departmentId, `referenceNumber` (unique), requestTypeId, serviceDeskId, requesterId, priority, `status` (String, defaults `SUBMITTED`), assignedToId/assignedTeam, parentRequestId, isConfidential, tags, `customFields` (Json), `formConfigSnapshot` (Json, P5-04), SLA timestamps (`slaDueAt`, `slaPausedAt`, `slaPauseDurationMs`, `firstResponseAt`, `resolvedAt`, `closedAt`, `completedAt`), `version` (optimistic concurrency). Relations: requestType, serviceDesk, requester, assignedTo, activities, attachments, approvals, approvalInstances, ITHardwareRequest, HRLeaveRequest, FinanceExpenseReimbursement, onboardingRequest, offboardingRequest, parent/child requests, assetAssignments, participants

Model	Purpose / key FKs
`RequestActivity`	Status-change / activity log (activityType, message, isSystemGenerated)
`RequestParticipant`	Users following/participating on a request
`RequestAttachment`	Uploaded files (S3)
`ITHardwareRequest` / `HRLeaveRequest` / `FinanceExpenseReimbursement`	Domain-specific detail rows on a Request
`RequestStatusDefinition`	DB-driven status catalog: `code` (unique), label, description, `category`, `displayOrder`, `isActive`, `lifecycleType`, `retiredAt` — the runtime status registry
`WorkflowFormSchema`	Versioned form schemas for request types

(c) Workflow-designer engine

Model	Purpose / key FKs
`WorkflowType`	A workflow (name, unique `code`). Children: steps, versions, requestTypes
`WorkflowStep`	Legacy step list (label, status, icon, displayOrder, isInitial, isFinal, `slaPause`) — per WorkflowType
`WorkflowVersion`	A published version of a workflow: `version` Int, `status` (DRAFT/...), `publishedAt`, `publishedById`. Children: nodes, edges
`WorkflowNode`	A status node in a versioned graph: `type` (WorkflowNodeType=STATUS), `statusCode`, label, displayOrder, positionX/Y, `isInitial`, `isFinal`, `slaPause`, icon, `config` (Json)
`WorkflowEdge`	A transition between nodes: fromNodeId→toNodeId, `transitionLabel`, `requiresComment`, `autoAssignRole`, `autoAssignUserId`, `allowedRoles`, `allowedExecutiveRoles`, `config` (Json)
`WorkflowTransition`	**DB transition table (runtime-authoritative):** fromStatus→toStatus, transitionLabel, requiresComment, autoAssignRole/UserId, tenantId, workflowTypeId, `allowedRoles`, `allowedExecutiveRoles`, `isActive`
`RequestStatusDefinition`	(see above) — DB status registry

> **Runtime source of truth:** the `WorkflowTransition` DB table + published `WorkflowNode`/`WorkflowEdge` graphs. `backend/src/utils/workflowTransitions.ts` (`VALID_TRANSITIONS`) is a seed/documentation fallback only.

(d) Approvals

Model	Purpose / key FKs
`RequestApproval`	Per-request approval step: `approverType` (CEO/HIRING_MANAGER/ENTITY), `approverId`, `entityId`, `policyId`, `stepOrder`, delegation fields (`delegatedBy/To/At`), reminders, `dueAt`, `status` (ApprovalStatus), comments

Model	Purpose / key FKs
`ApprovalInstance`	Runtime approval run for a request: `policyVersionId`, `status` (ApprovalStepStatus), steps[]
`ApprovalPolicy`	A policy bound to a requestType: name, `priority` (lower=higher, first match wins), `isActive`. Children: steps, versions
`ApprovalPolicyStep`	A step within a policy (role, timeoutHours)
`ApprovalPolicyVersion`	Versioned policy snapshot
`ApprovalDelegation`	Approval delegation: approvalId, fromUserId→toUserId, reason

(e) SLA & escalation

Model	Purpose / key FKs
`EscalationRule`	Per requestType: `triggerHoursAfterBreach`, `notifyRoles`, `label`, `isActive`. FK: requestTypeId
(SLA hours live on `RequestType.slaHours`; breach/escalation evaluated by `sla.service.ts` `checkEscalations()`; pause/resume via `sla-pause.service.ts`)	

(f) Notifications

Model	Purpose / key FKs
`Notification`	Per-user notification: `channel`, `status` (PENDING/SENT/...), subject, body, `relatedRequestId`, `deliveryId` (unique, links to NotificationDelivery), readAt, errorMessage
`NotificationDelivery`	Durable delivery record (outbox)

(g) IT Asset Management (ITAM)

Model	Purpose / key FKs
`Asset`	IT asset registry (LAPTOP/DESKTOP/MONITOR/PERIPHERAL/PHONE/NETWORK/PRINTER/SOFTWARE_LICENSE/OTHER), lifecycle status
`AssetAssignment`	Assignment of an asset to a user/request (tracking + lifecycle)

(h) Knowledge base

Model	Purpose / key FKs
`KnowledgeBaseArticle`	KB article (per desk via `ServiceDesk`), plus KB category/version models

(i) Other ESM support

`Department`, `Entity`, `Announcement`, `SystemSetting`, `AuditLog`, `Tenant`, `ReferenceNumber` (sequence), `NotificationTemplate`, `BannerConfig`, `RequestTypeEntityRouting` (entity routing — which entity/team handles a request).

2. Request status enum (`RequestStatus`, ~110 values)

Grouped by workflow domain:

- **Generic / IT simple:** SUBMITTED, IN_REVIEW, IN_PROGRESS, ACTION_REQUIRED, WAITING, APPROVED, REJECTED, CANCELLED, RESOLVED, COMPLETED.
- **HR recruitment / ESM travel (shared approval):** PENDING_CEO_APPROVAL, CEO_APPROVED/REJECTED, PENDING_GROUP_DCEO_APPROVAL, GROUP_DCEO_APPROVED/REJECTED, JOB_POSTED, PENDING_MANAGER_REVIEW, MANAGER_APPROVED, INTERVIEW_SCHEDULED, INTERVIEW_FEEDBACK_PENDING, CANDIDATE_REJECTED_INTERVIEW, HR_SCREENING, LOA_PENDING_APPROVAL/LOA_APPROVED/LOA_ISSUED/LOA_ACCEPTED/LOA_REJECTED.
- **IT procurement / hardware:** ACKNOWLEDGED_IT, PENDING_MANAGER_APPROVAL_IT, MANAGER_APPROVED_IT/REJECTED_IT, PENDING_VP_APPROVAL_IT, VP_APPROVED_IT/REJECTED_IT, PROCUREMENT_IN_PROGRESS, HARDWARE_ORDERED/RECEIVED, SOFTWARE_PROVISIONED, PENDING_CEO_APPROVAL_IT, CEO_APPROVED_IT/REJECTED_IT, PENDING_CTO_APPROVAL_IT, CTO_APPROVED_IT/REJECTED_IT, PENDING_INVOICE_IT, PENDING_CFO_APPROVAL_IT, CFO_APPROVED_IT/REJECTED_IT, PAYMENT_PROCESSING_IT, PAYMENT_DONE_IT, PENDING_DELIVERY_IT.
- **Finance:** PENDING_MANAGER_APPROVAL_FIN, MANAGER_APPROVED_FIN/REJECTED_FIN, PENDING_FINANCE_HEAD_APPROVAL, FINANCE_HEAD_APPROVED/REJECTED, PAYMENT_PROCESSING, PAYMENT_COMPLETED, REIMBURSEMENT_CLOSED, PENDING_CEO_APPROVAL_FIN, CEO_APPROVED_FIN/REJECTED_FIN; purchase requisition: FINANCE_PENDING_ACK, FINANCE_ACKNOWLEDGED, FINANCE_IN_PROGRESS, PENDING_CFO_APPROVAL_FIN, CFO_APPROVED_FIN/REJECTED_FIN, PENDING_GROUP_DCEO_APPROVAL, GROUP_DCEO_APPROVED/REJECTED, PAYMENT_PROCESSING_FIN, AWAITING_PAYMENT_CONFIRMATION, PAYMENT_CONFIRMED_FIN, TICKET_CLOSED_FIN.
- **Inter-company chargeback:** PENDING_FROM_ENTITY_APPROVAL, FROM_ENTITY_APPROVED/REJECTED, PENDING_TO_ENTITY_APPROVAL, TO_ENTITY_APPROVED/REJECTED, CHARGEBACK_FINANCE_REVIEW, AWAITING_CHARGEBACK_CONFIRMATION, CHARGEBACK_COMPLETED.
- **Onboarding:** ONBOARDING_SUBMITTED, ONBOARDING_PENDING_HR_APPROVAL, ONBOARDING_PRE_ARRIVAL_SETUP, ONBOARDING_READY_FOR_DAY_1, ONBOARDING_DAY_1_ORIENTATION, ONBOARDING_WEEK_1_INTEGRATION, ONBOARDING_MONTH_1/2/3_MILESTONE, ONBOARDING_COMPLETED.
- **Offboarding:** OFFBOARDING_SUBMITTED, OFFBOARDING_NOTICE_PERIOD, OFFBOARDING_KNOWLEDGE_TRANSFER, OFFBOARDING_FINAL_WEEK, OFFBOARDING_EXIT_PROCEDURES, OFFBOARDING_COMPLETED.

`RequestStatusLifecycleType`**:** OPEN / RESOLVED / CLOSED / CANCELLED — classifies which lifecycle bucket a status belongs to (drives SLA/terminal handling).

3. Integrity / behavior invariants

- **Runtime transitions are DB-driven.** The `WorkflowTransition` table (and published node/edge graphs) define what status moves are allowed; `requestTransition.service.ts` validates DB-first and only falls back to the `VALID_TRANSITIONS` map when the DB is empty. Editing `VALID_TRANSITIONS` alone does **not** change runtime behavior.
- **Terminal status timestamps:** transitions into terminal statuses set `resolvedAt`/`closedAt`/`completedAt` consistently (centralized in `requestTransition.service.ts`).
- **SLA pause/resume:** steps/transitions with `slaPause` auto-pause the SLA timer (`slaPausedAt` + `slaPauseDurationMs`); resumed on exit.
- **Form snapshot (P5-04):** `Request.formConfigSnapshot` + `formConfigVersion` freeze the form config at submission so later config edits don't rewrite historical requests.
- **Optimistic concurrency:** `Request.version` increments on writes; transactional workflow commands use version guards (409 on conflict).
- **Soft delete:** `deletedAt` on `Request` and others; hard deletes restricted.
- **Approval delegation is audited:** `RequestApproval` records `delegatedBy/To/At` and links `ApprovalDelegation` rows.

4. Working with the schema

- Migrations: `cd backend && npx prisma migrate dev` (local) — never destructive in production.
- After schema changes: `npm run prisma:generate`.
- Seed: `npm run prisma:seed` — seeds service desks, categories, request types, workflow types, transitions, SLA defaults, seed accounts.
- The seed also populates the `workflow_transitions` table that drives runtime validation.